

FLOYD, k -MÉMOÏDES ET APPLICATION AUX SÉQUENCES D'ARN : RAPPORT DE PROJET

Mercredi 3 décembre 2025

AKKAYA Tolunay
tolunay.akkaya@etu.univ-orleans.fr
Université d'Orléans

PAUL Nicolas
nicolas.paul1@etu.univ-orleans.fr
Université d'Orléans

Introduction

Ce rapport présente le détail d'un travail réalisé dans le cadre du cours de Programmation parallèle à l'Université d'Orléans. Le projet porte sur l'implantation d'algorithmes distribués appliqués sur des séquences d'acide ribonucléique (ARN) utilisant la bibliothèque Open MPI (Grabriel, 2004) et le langage C++ (ISO/IEC, 2011), en particulier, l'algorithme de Floyd-Warshall (Floyd, 1962) et l'algorithme Partitioning Around Medoids (PAM) (Kaufman, 1990).

Ce document commence donc par présenter notre travail d'implantation et nos méthodes d'analyses, pour ensuite illustrer et discuter les différents résultats et finir sur une conclusion du projet.

Méthodologie

La méthodologie présentée dans cette section décrit les différentes approches techniques que nous avons mises en œuvre pour paralléliser les algorithmes et analyser leur performance. Nous détaillons tout d'abord l'implantation de l'algorithme de Floyd-Warshall et sa construction à partir d'une topologie cartésienne. Nous présentons ensuite la méthode de parallélisation utilisée pour l'algorithme PAM. Enfin, nous expliquons la construction du graphe issu du séquençage ARN et de sa matrice adjacente.

Pour paralléliser l'algorithme de Floyd-Warshall, nous avons adopté une décomposition en blocs de la matrice de distances combinée à une topologie cartésienne bidimensionnelle en MPI. La matrice A de taille $n \times n$ est découpée en blocs carrés $b \times b$, chacun étant attribué à un processus au sein d'un communicateur cartésien 2D à partir duquel sont dérivés deux sous-communicateurs, l'un pour les lignes et l'autre pour les colonnes. À chaque itération associée à une valeur

intermédiaire k , les processus doivent obtenir les données nécessaires : dans chaque ligne, le processus possédant le bloc contenant la colonne k diffuse celui-ci via un `MPI_Bcast`, tandis que dans chaque colonne, le processus détenant le bloc de la ligne k effectue une diffusion équivalente. Après réception de ces blocs, chaque processus met à jour son bloc local sans autre synchronisation. Une fois toutes les itérations terminées, l'ensemble des blocs mis à jour est rassemblé pour reconstruire la matrice finale des plus courtes distances.

Pour l'implémentation de l'algorithme Partitioning Around Medoids (PAM), nous avons défini une structure dédiée permettant de stocker les médoïdes sélectionnés ainsi que les clusters qui leur sont associés. Le calcul du coût repose sur une fonction `compute_cost_local`, qui évalue la contribution locale d'un ensemble de médoïdes à partir des lignes de la matrice des distances. Afin de paralléliser cette étape, nous distribuons les lignes de la matrice entre les différents processus MPI : chacun calcule donc le coût local associé à un échange potentiel entre un médoïde et un nœud candidat. Une opération `MPI_AllReduce` est ensuite utilisée pour obtenir le coût global de chaque échange. À partir de ces coûts, les processus déterminent quels médoïdes doivent être retenus pour l'itération suivante.

Pour la construction du graphe à partir des données de séquences ARN, nous avons développé plusieurs fonctions dédiées. La fonction `lire_sequences` permet d'extraire l'ensemble des séquences présentes dans un fichier structuré donné en entrée. La comparaison entre deux séquences est effectuée par la fonction `hamming_distance_cutoff_raw`, qui calcule la distance de Hamming tout en appliquant un seuil prédéfini afin de déterminer si deux séquences doivent être considérées comme adjacentes dans le graphe. Enfin, la fonction `construire_matrice_adjacence_arn` utilise ces deux outils pour générer la matrice d'adjacence complète.

Pour analyser les performances de nos implémentations, nous avons effectué des mesures systématiques en répétant chaque exécution une dizaine de fois, puis en calculant la moyenne des temps observés afin de réduire l'impact des fluctuations liées au système. Les mesures temporelles reposent sur la commande UNIX `time`, dont nous avons retenu la valeur de *real time*, correspondant au temps total écoulé durant l'exécution parallèle. L'ensemble des tests a été réalisé sur les machines du laboratoire (salle E11), servant de référence commune : chaque nœud est équipé d'un processeur Intel Core i7-10700 à 2,90 GHz, comportant 8 cœurs physiques et 16 threads, et fonctionne sous Linux 5.15.0 avec Ubuntu 22.04.1 LTS. Les codes sources ont été compilés avec le G++ Compiler en version 11.2.0. Dans un premier temps, nous avons évalué les performances en exploitant les 16 threads disponibles sur une seule machine, puis nous avons étendu les expérimentations à une grappe de quatre machines interconnectées avec TCP (Eddy, 2022), pour un total de 64 cœurs.

Résultats

Dans cette section, nous présentons les performances de notre implémentation parallèle appliquée aux données de séquences ARN, en évaluant notamment l'impact des paramètres ε (seuil de similarité) et k (nombre de médoïdes). Nous analysons comment la variation de ces paramètres influence la vitesse de calcul du programme, permettant d'identifier les configurations offrant de meilleurs compromis entre temps d'exécution et qualité des clusters.

ε	k	np = 4 (secondes)	np = 16 (secondes)
50	4	9,581	10,451
70	4	11,518	13,095
90	4	11,370	12,994

Fig. 1 Évolution du temps d'exécution en variant ε .

Pour analyser l'impact des paramètres sur les performances, nous avons fait varier le seuil de similarité afin d'observer l'évolution du temps d'exécution, comme illustré à la **Fig. 1**. De manière analogue, nous avons étudié l'influence du nombre de médoïdes, présentée à la **Fig. 2**. Dans les deux cas, on constate que le temps d'exécution augmente avec la valeur du paramètre considéré, montrant une tendance ascendante : plus ε ou k est élevé, plus le programme nécessite de temps pour effectuer les calculs et construire les clusters correspondants.

ε	k	np = 4 (secondes)	np = 16 (secondes)
70	3	11,339	12,985
70	5	11,648	13,487
70	7	12,033	14,331

Fig. 2 Évolution du temps d'exécution en variant k .

Il n'a pas été possible d'effectuer des exécutions au-delà de np = 16 sur la grappe de machines du laboratoire. En effet, les communications entre nœuds via TCP se sont révélées instables et peu reproductibles : certaines exécutions entraînaient des crashes inexpliqués, bien que de manière sporadique. Ces interruptions étaient liées au runtime d'Open MPI, qui ne parvenait pas à gérer correctement certaines situations de communication distribuée sur ce réseau. Nous avons donc limité les expérimentations

à des configurations stables, évitant ainsi de fausser les temps d'exécution rapportés.

En conclusion, nos observations montrent que les paramètres ε et k ont un impact direct sur le temps d'exécution du programme parallèle, même si, dans le cadre de nos expérimentations, ces variations restent de l'ordre de quelques secondes. Toutefois, nous supposons que sur des jeux de données beaucoup plus volumineux, ces différences deviendraient beaucoup plus marquées, soulignant l'importance du choix judicieux des paramètres afin d'optimiser l'efficacité des algorithmes parallèles.

Conclusion

Ce projet de programmation parallèle a permis d'explorer la mise en œuvre et l'optimisation de deux algorithmes essentiels, Floyd-Warshall et PAM, ainsi que la construction de graphes à partir de séquences ARN. Nous avons pu développer des versions parallèles adaptées à une architecture à mémoire distribuée et analyser leurs performances sur différentes configurations de calcul. L'étude des paramètres, tels que le seuil ε et le nombre de médoïdes k , a montré leur influence sur les temps d'exécution et sur la structure des partitions obtenues. Bien que les expérimentations aient été limitées par les contraintes matérielles et réseau du laboratoire, le projet illustre clairement les bénéfices de la parallélisation pour traiter des problèmes de grande échelle.

Références

Gabriel, E., Fagg, G. E., Bosilca, G., Angskun, T., Dongarra, J. J., Squyres, J. M., ... & Woodall, T. S. (2004). Open MPI: Goals, concept, and design of a next generation MPI implementation. Dans *European Parallel Virtual Machine/Message Passing Interface Users' Group Meeting* (pp. 97-104). Berlin, Heidelberg: Springer Berlin Heidelberg.

ISO/IEC. (2011). ISO/IEC 14882:2011 – Information technology – Programming languages – C++. *International Organization for Standardization*.

Floyd, R. W. (1962). Algorithm 97: shortest path. *Communications of the ACM*, 5(6), 345-345.

Kaufman, L. (1990). Partitioning around medoids (program pam). *Wiley series in probability and statistics*, 344, 68-125.

Eddy, W. (Ed.). (2022). RFC 9293: Transmission Control Protocol (TCP).